LAB 1

Title: String Manipulation and Identifier Validation in C.

Theory:

In C programming, a string is a sequence of characters stored in a contiguous block of memory and terminated by a null character ('\0'). String manipulation involves various operations on these sequences, such as copying, concatenating, comparing, and searching. Identifier validation is the process of checking if a given string conforms to the rules for naming variables, functions, and other program elements.

String Manipulation

C does not have a built-in string data type. Instead, strings are implemented as character arrays. The string.h library provides a set of standard functions for manipulating these arrays. The core idea behind C strings is that they are null-terminated, which allows functions to determine the end of the string without needing an explicit length parameter.

Common string manipulation functions include:

- strlen(str): Returns the length of the string, excluding the null terminator.
- strcpy(dest, src): Copies the string src to dest. Be cautious to ensure dest has enough allocated memory to prevent buffer overflow.
- strcat(dest, src): Concatenates src to the end of dest. Again, ensure dest is large enough.
- strcmp(str1, str2): Compares two strings. It returns an integer value:
    - 0 if the strings are identical.
    - A negative value if str1 comes before str2 in lexicographical order.
    - A positive value if str1 comes after str2.
- strstr(haystack, needle): Searches for the first occurrence of the substring needle within the string haystack. It returns a pointer to the beginning of the found substring or NULL if not found.

Identifier Validation

An identifier is a name used to identify a variable, function, or other program entity. C has strict rules for what constitutes a valid identifier. Validating a string as an identifier involves checking it against these rules. The rules for a valid C identifier are:

1. Starts with a letter or an underscore (_): The first character cannot be a digit.
2. Subsequent characters can be letters, digits, or underscores: After the first character, numbers are permitted.
3. Case-sensitive: myVar and myvar are considered different identifiers.
4. No special characters or spaces: Identifiers cannot contain symbols like !, @, #, $, %, or spaces.
5. Not a C keyword: The identifier cannot be one of C's reserved keywords, such as int, for, while, or return.

To implement identifier validation in C, you typically need to:

- Check the first character: Use functions from the <ctype.h> library like isalpha() and a check for _ to see if the first character is valid.
- Iterate through the rest of the string: Loop through the remaining characters and use isalnum() (which checks for either letters or digits) and a check for _ to ensure they are all valid.
- Check for length and other constraints: Although C standards don't specify a maximum length, most compilers have a limit
- Check against reserved keywords: This can be done by storing keywords in an array and using a function like strcmp to compare the input string against each keyword.

By combining string manipulation techniques with character-level validation from <ctype.h>, you can build a robust function to check if a string is a valid identifier.

Program 1: WAP to find prefix, suffix and substring from given string.

Source code:

```
#include <stdio.h>
#include <string.h>
int main()
{
   char str[100];
   int i, j, len;
   printf("Program by: Mahesh Udas\n");
   printf("Enter a string: ");
   scanf("%s", str);
   len = strlen(str);
   printf("\nPrefixes:\n");
   for (i = 1; i <= len; i++)
   {
      for (j = 0; j < i; j++)
      {
         printf("%c", str[j]);
      }
      printf("\n");
   }
   printf("\nSuffixes:\n");
   for (i = 0; i < len; i++)
   {
      for (j = i; j < len; j++)
      {
         printf("%c", str[j]);
      }
      printf("\n");
   }
   printf("\nSubstrings:\n");
   for (i = 0; i < len; i++)
   {
      for (j = i; j < len; j++)
      {
         for (int k = i; k <= j; k++)
         {
            printf("%c", str[k]);
         }
         printf("\n");
      }
   }
   return 0;
}
```

```
Program by: Mahesh Udas
Enter a string: mahesh

Prefixes:
m
ma
mah
mahe
mahes
mahesh

Suffixes:
mahesh
ahesh
hesh
esh
sh
h

Substrings:
m
ma
mah
mahe
mahes
mahesh
a
ah
ahe
ahes
ahesh
```

Output and Discussion:

This C program prints all prefixes, suffixes, and substrings of a user-input string. It uses nested loops to iterate through character positions and display combinations. The strlen function determines string length, and scanf captures input. It demonstrates string manipulation fundamentals and loop control in a clear, structured, and educational format.

Program 2: WAP to validate C identifiers and keywords Source code:

Source code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
const char *keywords[] = {
    "auto", "break", "case", "char", "const", "continue",
"default", "do", "double", "else", "enum", "extern",
"float", "for", "goto", "if", "int", "long", "register",
"return", "short", "signed", "sizeof", "static", "struct",
"switch", "typedef", "union", "unsigned", "void",
"volatile", "while"
};
int keywordCount = 32;
int isKeyword(const char *str)
{
   for (int i = 0; i < keywordCount; i++)
   {
      if (strcmp(str, keywords[i]) == 0)
         return 1;
   }
   return 0;
}
int isValidIdentifier(const char *str)
{
   if (!(isalpha(str[0]) || str[0] == '_'))
      return 0;
   for (int i = 1; str[i] != '\0'; i++)
   {
      if (!(isalnum(str[i]) || str[i] == '_'))
         return 0;
   }
   return 1;
}
int main()
{
   char str[50];
   printf("Program by: Mahesh Udas\n");
   printf("Enter a string: ");
   scanf("%s", str);
   if (isKeyword(str))
   {
      printf("'%s' is a C keyword.\n", str);
   }
   else if (isValidIdentifier(str))
   {
      printf("'%s' is a valid C identifier.\n", str);
   }
   else
   {
      printf("'%s' is NOT a valid C identifier.\n", str);
   }
   return 0;
}
```

Output and discussion

```
Program by: Mahesh Udas
Enter a string: Mahesh
'Mahesh' is a valid C identifier.
```

```
Program by: Mahesh Udas
Enter a string: const
'const' is a C keyword.
```

```
Program by: Mahesh Udas
Enter a string: signed
'signed' is a C keyword.
```

This C program checks whether a user-input string is a C keyword or a valid identifier. It compares the string against a list of 32 keywords and verifies identifier rules using character checks. It demonstrates string handling, validation logic, and conditional output, helping users understand lexical analysis and basic syntax rules in C programming.

Conclusion:

This lab successfully demonstrated fundamental string manipulation and identifier validation in C programming. Through two practical programs, we explored essential string operations including prefix/suffix generation and substring extraction using nested loops and the strlen() function. The identifier validation program effectively implemented C's naming rules by checking first character validity, subsequent character constraints, and keyword conflicts using ctype.h functions and string comparison. These exercises reinforced core concepts of null-terminated strings, character arrays, loop structures, and lexical analysis fundamentals. The lab provided hands-on experience with string handling techniques crucial for understanding C programming and compiler design principles.